Zellic

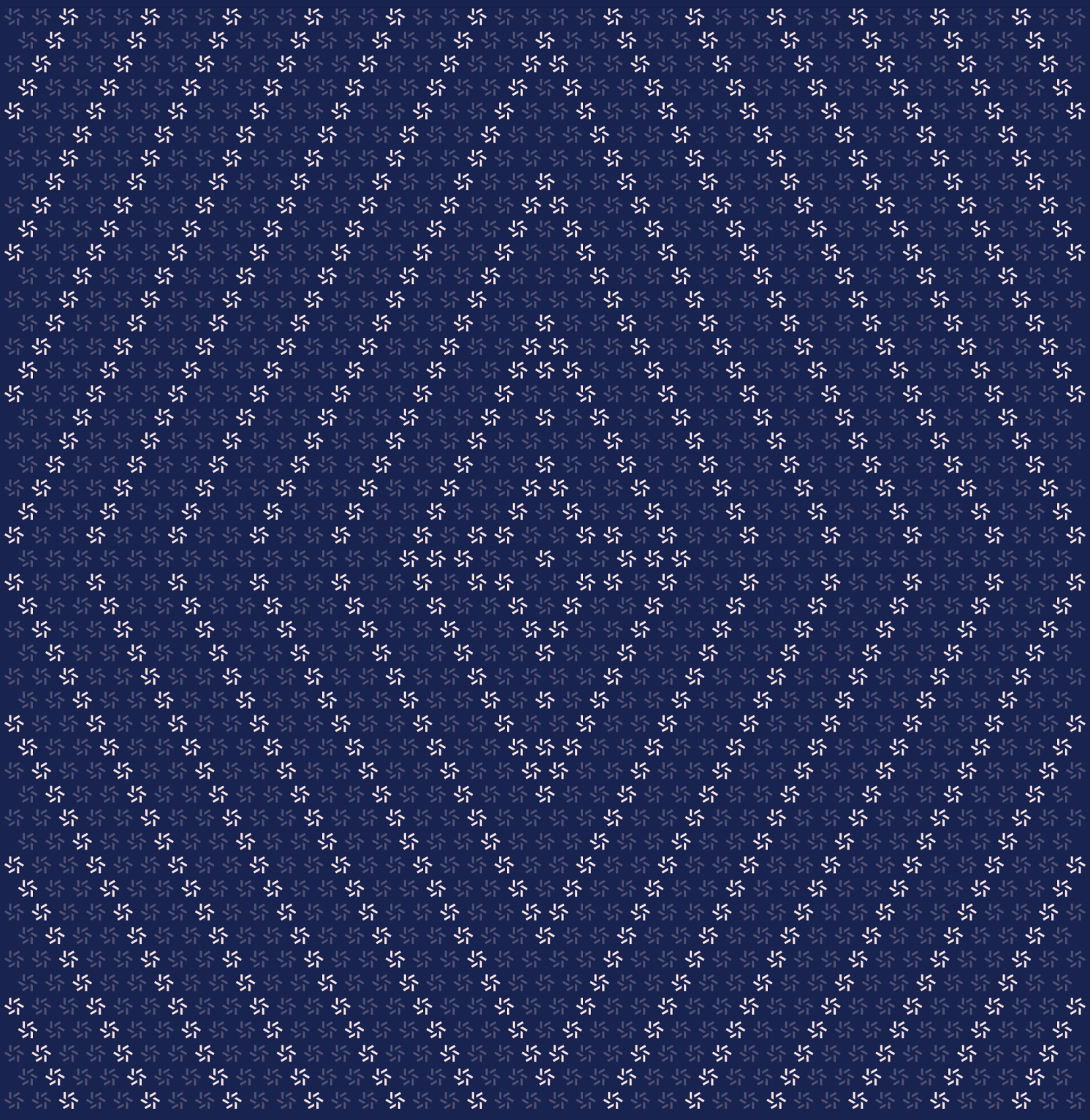
Prepared for
Luka Avbreht
Flare Foundation

Prepared by
Nipun Gupta
Weipeng Lai
Zellic

June 22, 2026

Flare FAsset and Smart Account

Smart Contract Patch Review



Contents

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Introduction

From June 12th to June 17th, 2026, we were asked to review a few minor patches to Flare FAsset and Smart Account that added the following changes to the two repos:

1. On smart-accounts, the new `0xFE` memo opcode lets the 42-byte XRPL memo carry only `[0xFE][walletId][fee][keccak256(_data)]`; the actual `PackedUserOperation` is delivered out-of-band via the AssetManager-forwarded `_data` parameter.
2. On FAssets, a new direct-minting entry point on DirectMintingFacet accepts that additional bytes payload and forwards it to the smart account, so memo opcodes that do not fit in the XRPL memo can still receive their inputs.
3. The remaining changes are direct-minting delay fixes, an emergency-pause access-control split, multiple CoreVaultManager unpause senders, a governance-configurable `executorChangeAfterSeconds`, and a `CREATE2`-stability fix for personal accounts.

1.1. Results

During our assessment on the scoped Flare FAsset and Smart Account contracts, we discovered one finding, which was of high impact.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	1
Medium	0
Low	0
Informational	0

1.2. Scope

The engagement involved a review of the following targets:

Flare FAsset and Smart Account Contracts

Type	Solidity
Platform	EVM-compatible

Target	Primary Review
Repository	https://github.com/flare-foundation/fassets ↗
Version	Diff between commit e3ded6b5330401a7b82880ed1804c93279ddbe4f and cc71f1dd7f5c25bf2f419d418432eabc6f20a9de
Programs	contracts/assetManager/diamondInitializers/DirectMintingAndRedeemExtendedInit.sol contracts/assetManager/facets/AgentCollateralFacet.sol contracts/assetManager/facets/AgentVaultAndPoolSupportFacet.sol contracts/assetManager/facets/ChallengesFacet.sol contracts/assetManager/facets/DirectMintingFacet.sol contracts/assetManager/facets/DirectMintingSettingsFacet.sol contracts/assetManager/facets/EmergencyPauseFacet.sol contracts/assetManager/facets/LiquidationFacet.sol contracts/assetManager/facets/MintingDefaultsFacet.sol contracts/assetManager/facets/MintingFacet.sol contracts/assetManager/facets/RedeemExtendedSettingsFacet.sol contracts/assetManager/facets/RedemptionConfirmationsFacet.sol contracts/assetManager/facets/SettingsManagementFacet.sol contracts/assetManager/library/Agents.sol contracts/assetManager/library/DirectMinting.sol contracts/assetManager/library/Minting.sol contracts/assetManager/library/RedemptionRequests.sol contracts/assetManager/library/Redemptions.sol contracts/assetManager/library/TransactionAttestation.sol contracts/assetManager/library/data/MintingRateLimiter.sol contracts/assetManagerController/implementation/AssetManagerController.sol contracts/collateralPool/implementation/CollateralPool.sol contracts/coreVaultManager/implementation/CoreVaultManager.sol contracts/diamond/library/LibDiamond.sol contracts/fassetToken/implementation/CheckPointable.sol contracts/mintingTagManager/MintingTagManager.sol contracts/mintingTagManager/MintingTagManagerProxy.sol

Target	Primary Review
Repository	https://github.com/flare-foundation/flare-smart-accounts ↗
Version	`Diff between commit c9dd523f8595a6f8424b0b1f39236668a034033b and 39d68430430b933f335607e839091b649a4b4ab5`

Programs	`contracts/composer/implementation/FAssetRedeemComposer.sol`
	`contracts/smartAccounts/facets/MemoInstructionsFacet.sol`
	`contracts/smartAccounts/library/MemoInstructions.sol`
	`contracts/smartAccounts/library/PersonalAccounts.sol`
	`contracts/smartAccounts/library/Vaults.sol`

Target	Secondary Review
Repository	https://github.com/flare-foundation/fassets ↗
Version	`Diff between commit cc71f1dd7f5c25bf2f419d418432eabc6f20a9de and 9400bc86a64e689cb6c94c7c15fcee6dc6d11ab3`

Programs	`contracts/assetManager/facets/CoreVaultClientFacet.sol`
	`contracts/assetManager/facets/DirectMintingFacet.sol`
	`contracts/assetManager/facets/TransferToCoreVaultFacet.sol`
	`contracts/collateralPool/implementation/CollateralPool.sol`

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Nipun Gupta
Engineer
nipun@zellic.io ↗

Weipeng Lai
Engineer
weipeng.lai@zellic.io ↗

1.3. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.

2. Detailed Findings

2.1. CR might not be preserved due to system redemption fee

Target	contracts/assetManager/library/RedemptionRequests.sol		
Category	Business Logic	**Severity**	High
Likelihood	Medium	**Impact**	High

Description

The patch in the FAssets file contracts/assetManager/library/RedemptionRequests.sol adds a new system redemption fee that is subtracted from `_args.valueAMG` while creating the redemption request in the `createRedemptionRequest` function.

The `selfCloseExit` and `selfCloseExitTo` functions calculate `requiredFAssets` through `_getFAssetRequiredToNotSpoilCR`, assuming the full FAsset amount redeemed by the pool will reduce the agent's backed FAsset amount. However, when the self-close path uses system redemption, only `requiredFAssets - systemFee` is applied as the actual redemption amount, while `systemFee` is reminted. Due to this, the pool-backed FAsset debt is reduced by less than the amount used in the collateral-ratio calculation. When the pool is already below the exit-collateral ratio, this causes `selfCloseExit` to reduce the pool-collateral ratio even though the function is intended to preserve it.

The following test demonstrates this:

```
it("PoC: system fee makes below-exit-CR self-close exit reduce pool collateral
    ratio", async () => {
    await enableSystemRedemptionFee();
    const poolExitCollateralRatioBIPS = toBIPS(3.5);
    const targetPoolCollateralRatioBIPS = toBIPS(3.0);
    const agent = await Agent.createTest(context, agentOwner1,
    underlyingAgent1, {
        poolExitCollateralRatioBIPS,
    });
    const minter = await Minter.createTest(context, minterAddress1,
    underlyingMinter1, context.underlyingAmount(1e8));
    const fullAgentCollateral = toWei(1e7);
    await agent.depositCollateralsAndMakeAvailable(fullAgentCollateral,
    fullAgentCollateral);
    await minter.performMinting(agent.vaultAddress, 300);
    const minterPoolDeposit = toWei(90000);
    await agent.collateralPool.enter({ from: minter.address, value:
    minterPoolDeposit });
```

```
    const minterPoolTokensWei = await
    agent.collateralPoolToken.balanceOf(minter.address);
    await context.fAsset.approve(agent.collateralPool.address,
    context.convertLotsToUBA(300), { from: minter.address });
    await agent.setPoolCollateralRatioByChangingAssetPrice
    (targetPoolCollateralRatioBIPS);

    const infoBefore = await agent.getAgentInfo();
    assert.isTrue(
        toBN(infoBefore.poolCollateralRatioBIPS).lt(toBN(infoBefore.
        poolExitCollateralRatioBIPS)),
        `expected pool CR below exit CR before self-close exit,
    poolCR=${infoBefore.poolCollateralRatioBIPS},
    exitCR=${infoBefore.poolExitCollateralRatioBIPS}`);

    const requiredFAssets = await
    agent.collateralPool.fAssetRequiredForSelfCloseExit(minterPoolTokensWei);
    assert.isTrue(requiredFAssets.gte(await context.assetManager.lotSize()));

    await time.deterministicIncrease(await
    context.assetManager.getCollateralPoolTokenTimelockSeconds());
    const response = await
    agent.collateralPool.selfCloseExit(minterPoolTokensWei, false,
    underlyingMinter1, ZERO_ADDRESS,
        { from: minter.address });
    const feePaid = requiredEventArgsFrom(response, context.assetManager,
    'SystemRedemptionFeePaid');
    assert.isTrue(toBN(feePaid.feeUBA).gtn(0));

    const infoAfter = await agent.getAgentInfo();
    console.log('infoAfter', infoAfter.poolCollateralRatioBIPS);
    console.log('infoBefore', infoBefore.poolCollateralRatioBIPS);
    assert.isTrue(
        toBN(infoAfter.poolCollateralRatioBIPS).lt(toBN(infoBefore.
        poolCollateralRatioBIPS)),
        `expected system fee to reduce pool CR,
    before=${infoBefore.poolCollateralRatioBIPS},
    after=${infoAfter.poolCollateralRatioBIPS}`);
});
```

In this example, due to the system redemption fee, the collateral ratio is decreased from 30000 to 29997 when it should be preserved.

Impact

The `selfCloseExit` and `selfCloseExitTo` functions can worsen the pool-collateral ratio when the system redemption fee is enabled and the exit is settled through underlying redemption. This breaks the invariant that self-close exits should preserve the pool-collateral ratio when the pool is already below the exit-collateral ratio. The collateral ratio is used to determine whether the pool collateral is underwater for liquidation, so if the pool-collateral ratio is already close to the pool-collateral type's minimum collateral ratio, repeated affected self-close exits can make liquidation start sooner than expected. The degraded collateral ratio also leaves the remaining pool participants with a larger share of the pool's collateral shortfall.

Recommendations

We recommend accounting for the system redemption fee when calculating the FAsset amount required for self-close exits.

Remediation

This issue has been acknowledged by Flare Foundation, and fixes were implemented in the following commits:

- `9cf653d9` ↗
- `62435852` ↗

3. Patch Review

This section documents notable and minor changes applied to the in-scope smart contracts across two repositories:

- `fasset`, between commit e3ded6b5 ↗ and commit cc71f1dd ↗.
- `flare-smart-accounts`, between commit c9dd523f ↗ and commit 39d68430 ↗.

3.1. Notable changes

The following are notable changes made to the codebases.

`fasset`

- **Direct minting can now forward extra data to Smart Accounts.** `executeDirectMintingWithData()` was added as a direct-minting entry point that forwards `_data` to the configured Smart Account manager through `handleMintedFAssets()`. Extra data is only allowed for minting to Smart Accounts; direct recipient minting rejects it.
- **Delayed direct minting now snapshots the mint target.** Delayed minting records whether the mint is for a Smart Account, the recipient, and the allowed executor at delay creation time. This prevents later tag transfers or executor changes from changing where a delayed direct mint resolves.
- **Native executor value is returned when direct minting is delayed.** If rate limits or large-minting rules delay execution, the native value sent by the executor is returned immediately because a different executor may later execute the delayed mint.
- **A system redemption fee was added.** Governance can configure `systemRedemptionFeeBIPS` and `systemRedemptionFeeReceiver`. For ordinary redemption requests, the fee is subtracted from the redeemed amount and reminted to the fee receiver; core-vault transfers are excluded.
- **Emergency pause permissions were split into pause and unpause powers.** Asset Manager emergency pause calls now use permission flags instead of a governance boolean. The controller tracks separate emergency pause and emergency unpause senders.
- **Minting tag executor changes became configurable and safer on transfer.** `MintingTagManager` now has governance-configurable `executorChangeAfterSeconds`, can cancel pending executor changes by reverting to the active executor, and clears active and pending executor state immediately on tag transfer.

`flare-smart-accounts`

- **The direct-mint callback was renamed and extended.** `mintedFAssets()` was replaced by `handleMintedFAssets()`, adding a `_data` argument so Asset Manager direct minting can pass data that is not carried directly in the XRPL memo.
- **A `0xFE` memo instruction mode was added.** Memo instruction `0xFE` carries a compact memo containing a hash of the full ABI-encoded `PackedUserOperation`; the full payload is supplied in `_data` and verified with `keccak256(_data)`. Existing `0xFF` inline-user-

operation memo execution remains supported.

- **Personal Account `CREATE2` derivation was frozen against metadata drift.** `PersonalAccounts` now uses a constant `PROXY_CREATION_CODE` instead of `type(PersonalAccountProxy).creationCode`, preserving predicted Personal Account addresses across rebuilds, compiler metadata changes, and supported chains.

3.2. Minor changes

The following are minor changes made to the codebases.

`fasset`

- `setDirectMintingFeeReceiver()` now rejects the zero address so direct minting cannot be accidentally broken by clearing the fee receiver.
- `AssetManagerController.updateContracts()` is now restricted to immediate governance.
- `RedemptionConfirmationsFacet` now validates redemption payment sufficiency against `intendedReceivedAmount`, so blocked-payment handling is based on the intended amount rather than the actually received amount.
- `MintingRateLimiter` now widens timestamp and window-size arithmetic to `uint256` before multiplication.
- Executor fees sent to the burn address now use `Transfers.transferNAT()`.

`flare-smart-accounts`

- `InvalidInstructionType` now includes both the provided instruction type and the vault's expected type.
- The FAsset redeem composer change is limited to preserving `_guid` in a local variable to avoid a stack-too-deep compilation issue.